Copyright@LAMDA
Please contact the author for non-educational use

Zhi-Hua Zhou

Machine Learning

Translated by Shaowu Liu

The slides of this chapter
are translated by Zheng-Mao Zhu

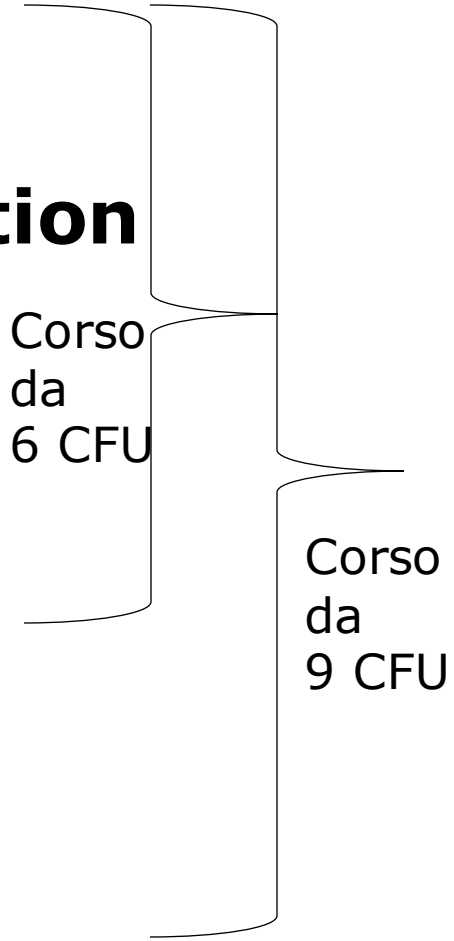


Springer

Chapter 7
Bayes Classifiers

Chapter List

- Bayes Decision Theory
- Maximum Likelihood Estimation
- Naïve Bayes Classifier
- Semi-Naïve Bayes Classifier
- Bayesian Network
- EM Algorithm



Chapter List

- Bayes Decision Theory
- Maximum Likelihood Estimation
- Naïve Bayes Classifier
- Semi-Naïve Bayes Classifier
- Bayesian Network
- EM Algorithm

Bayes Decision Theory

- Bayesian decision theory is a fundamental decision-making approach under the probability framework.
  - How to make decisions in the presence of uncertainty
    - Bayesian Decision Theory was motivated by problems arising during the 2nd World War: radar for aircraft detections, code-breaking and decryption.
      The task is to estimate the state but we only have a noisy, or corrupted, observation.
    - Today, it finds applications is finance, insurance and gambling
      - Let R be a set of basic rewards $\{r_1, r_2, ..r_N\}$ for the N outcomes of a random event having probabilities $\{p_1, p_2, ..p_N\}$ with $1 = \sum_i p_i$.
      - The expected reward for a gambling is:

$$\sum_{i=1}^{N} r_i \cdot p_i$$

  - When all relevant probabilities were known, Bayesian decision theory makes optimal classification decisions based on the probabilities and costs (instead of rewards in this task domain).

Bayes Decision Theory in classification

- Let us assume that there are N distinct class labels, that is, $y = \{c_1, c_2, \ldots, c_N\}$. Let λ_{ij} denote the cost of misclassifying a sample of class c_j as class c_i. Then, with the posterior probability $P(c_i \mid \boldsymbol{x})$ we can calculate the **expected loss** of classifying a sample $\mathbf{x}$ as class c_i, that is, the conditional risk of the sample $\mathbf{x}$:

It is the expected cost of classifying examples in class c_i, having considered all the possible classes as the true class

$$R(c_i \mid \mathbf{x}) = \sum_{j=1}^{N} \lambda_{ij} P(c_j \mid \mathbf{x}) \qquad (7.1)$$

$$\lambda_{ij} = \begin{cases} 0, & \text{if } i = j; \\ \lambda_{ij} & \text{otherwise,} \end{cases}$$

- Our task is to find a decision rule $h : X \mapsto Y$ that minimizes the overall risk:

$$R(h) = \mathbf{E}_x \left[R(h(\mathbf{x}) \mid \mathbf{x}) \right] \qquad (7.2)$$

Bayes Decision Theory in classification

- The overall risk $R(h)$ is minimized when the conditional risk $R(h(\mathbf{x}) \mid \mathbf{x})$ of each sample $\mathbf{x}$ is minimized.

Bayes Decision Theory in classification

- The overall risk $R(h)$ is minimized when the conditional risk $R(h(\mathbf{x}) \mid \mathbf{x})$ of each sample $\mathbf{x}$ is minimized.

- This leads to the Bayes decision rule: to minimize the overall risk, classify each sample as the class that minimizes the conditional risk $R(c \mid \mathbf{x})$

$$h^*(x) = \underset{c \in y}{\operatorname{argmin}} R(c \mid x) \qquad (7.3)$$

  - where h^* is called the Bayes optimal classifier, and its associated overall risk $R(h^*)$ is called the Bayes risk.
  - $1 - R(h^*)$ is the best performance that can be achieved by any classifiers, that is, the theoretically achievable upper bound of accuracy for any machine learning models.

Bayes Decision Theory in classification

- To be specific, if the objective is to minimize the misclassification rate, then the misclassification loss λ_{ij} can be written as

$$\lambda_{i,j} \begin{cases} 0, & \text{if } i = j; \\ 1, & \text{otherwise}, \end{cases} \qquad (7.4)$$

Bayes Decision Theory in classification

- To be specific, if the objective is to minimize the misclassification rate, then the misclassification loss λ_{ij} can be written as

$$\lambda_{i,j} \begin{cases} 0, & \text{if } i = j; \\ 1, & \text{otherwise}, \end{cases} \qquad (7.4)$$

- and the conditional risk is

$$R(c \mid \mathbf{x}) = 1 - P(c \mid \mathbf{x}) \qquad (7.5)$$

where c is the correct class for the example $\boldsymbol{x}$.

Bayes Decision Theory in classification

- To be specific, if the objective is to minimize the misclassification rate, then the misclassification loss λ_{ij} can be written as

$$\lambda_{i,j} \begin{cases} 0, & \text{if } i = j; \\ 1, & \text{otherwise}, \end{cases} \tag{7.4}$$

- and the conditional risk is

$$R(c \mid \mathbf{x}) = 1 - P(c \mid \mathbf{x}) \tag{7.5}$$

where c is the correct class for the example $\boldsymbol{x}$.

- Then, the Bayes optimal classifier that minimizes the misclassification rate is

$$h^*(x) = \underset{c \in y}{\operatorname{argmax}} P(c \mid x) \tag{7.6}$$

which classifies each sample $\mathbf{x}$ as the class that maximizes its posterior probability $P(c \mid \mathbf{x})$.

Bayes Decision Theory in classification

- We can see that the Bayes decision rule relies on the posterior probability $P(c \mid \mathbf{x})$.

- However, it's often difficult to obtain in practice. The task of machine learning is then to accurately estimate the posterior probability $P(c \mid \mathbf{x})$ from the finite training samples.
- Generally speaking, there are two strategies :
  - discriminative models
    - Given $\mathbf{x}$, predict C by estimating $P(c \mid \mathbf{x})$ directly.
    - Decision trees, BP neural networks and support vector machines.
  - generative models
    - estimate the joint probability $P(\mathbf{x}, c)$ first and then estimate $P(c \mid \mathbf{x})$
    - For generative models, we must evaluate:

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})} \qquad (7.7)$$

Bayes Decision Theory in classification

- For generative models, we must evaluate:

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})} \tag{7.7}$$

Bayes Decision Theory in classification

- For generative models, we must evaluate:

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})} \tag{7.7}$$

- According to Bayes' theorem,
  $P(c \mid \mathbf{x})$ can be written as:

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})} \tag{7.8}$$

Example of Bayes Theorem

- Given:
  - A doctor knows that meningitis causes stiff neck 50% of the time
  - Prior probability of any patient having meningitis is 1/50,000
  - Prior probability of any patient having stiff neck is 1/20

- If a patient has stiff neck, what's the probability he/she has meningitis?

$$P(M \mid S) = \frac{P(S \mid M)P(M)}{P(S)} = \frac{0.5 \times 1/50000}{1/20} = 0.0002$$

Bayes Decision Theory in classification

- For generative models, we must evaluate:

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})} \tag{7.7}$$

- According to Bayes' theorem, $P(c \mid \mathbf{x})$ can be written as:

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})} \tag{7.8}$$

the prior probability represents the proportion of each class in the sample, which can be estimated by the frequency of each class in the training set

Bayes Decision Theory in classification

- For generative models, we must evaluate:

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})} \tag{7.7}$$

- According to Bayes' theorem, $P(c \mid \mathbf{x})$ can be written as:

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})} \tag{7.8}$$

the prior probability represents the proportion of each class in the sample, which can be estimated by the frequency of each class in the training set

the evidence factor, which is independent of the class

Bayes Decision Theory in classification

- For generative models, we must evaluate:

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})} \tag{7.7}$$

- According to Bayes' theorem, $P(c \mid \mathbf{x})$ can be written as:

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})} \tag{7.8}$$

the class-conditional probability, also known as the **likelihood**, of the sample $\mathbf{x}$ with respect to class c

the prior probability represents the proportion of each class in the sample, which can be estimated by the frequency of each class in the training set

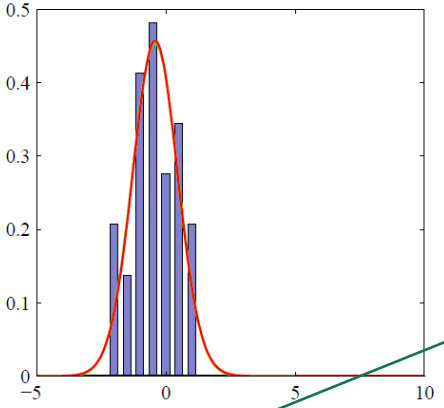
the evidence factor, which is independent of the class

Chapter List

- Bayes Decision Theory
- **Maximum Likelihood Estimation**
- Naïve Bayes Classifier
- Semi-Naïve Bayes Classifier
- Bayesian Network
- EM Algorithm

Maximum Likelihood Estimation

- A general strategy of estimating the class-conditional probability is to hypothesize a fixed form of probability distribution, and then estimate the distribution parameters using the training samples.

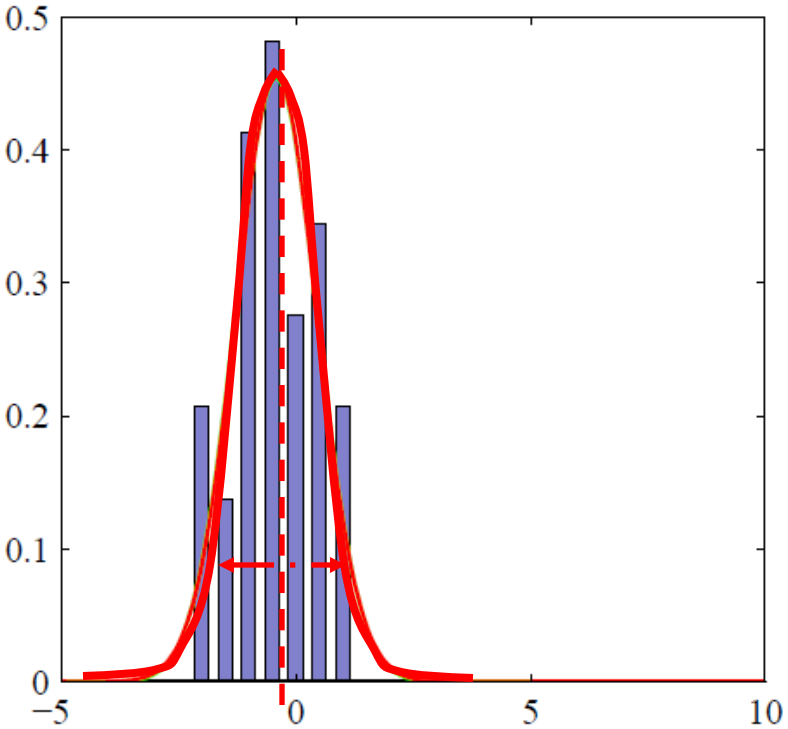


If the probability distribution takes the form of a Gaussian, the parameter vector $\boldsymbol{\theta}_c = \langle \mu_c, \sigma_c^2 \rangle$

- let $P(\mathbf{x} \mid c)$ denote class-conditional probability of class C,
  - suppose $P(\mathbf{x} \mid c)$ has a fixed form determined by a parameter vector $\boldsymbol{\theta}_c$. Then, the task is to estimate $\boldsymbol{\theta}_c$ from a training set D.

Maximum Likelihood Estimation

- Let us see in an animation, the task to estimate the likelihood of parameters θ_c from a training set D.



$\theta_c = \langle \mu_c = 0, {\sigma_c}^2 = 1.5 \rangle$

Are the parameters $\theta_c = \langle \mu_c = 2.3, {\sigma_c}^2 = 1.5 \rangle$ likely with these data?

Maximum Likelihood Estimation

- About the task to estimate θ_c from a training set D:
- The training process of probabilistic models is the process of parameter estimation. There are two different ways of thinking about parameters:
  - **(The Frequentist school)** Parameters have unknown but fixed values, and hence they can be determined by some approaches such as *optimizing the likelihood function*.
  - **(The Bayesian school)** Parameters are *unobserved random variables following some distribution*, and hence we can assume prior distributions for the parameters and estimate posterior distribution from observed data.

Maximum Likelihood Estimation

independent and identically distributed

- Let D_c denote the set of class c samples in the training set D, and further suppose the samples are i.i.d. samples. Then, the likelihood of D_c for a given parameter θ_c is:

$$P(D_c \mid \boldsymbol{\theta_c}) = \prod_{\mathbf{x} \in D_c} P(\mathbf{x} \mid \boldsymbol{\theta_c}) \qquad (7.9)$$

  - Applying the MLE to θ_c is about finding a parameter value $\hat{\theta}_c$ that maximizes the likelihood $P(D_c \mid \theta_c)$. Intuitively, the MLE aims to find a value of θ_c that maximizes the "likelihood" that the data will present.

For the Gaussian distribution of probability, MLE treats θ_c as the variables and x as a constant:

$$f_c(x; \theta_c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{-\frac{(x-\mu_c)^2}{2\sigma_c^2}}$$

Maximum Likelihood Estimation

- Let D_c denote the set of class c samples in the training set D, and further suppose the samples are i.i.d. samples. Then, the likelihood of D_c for a given parameter $\boldsymbol{\theta}_c$ is:

$$P(D_c \mid \boldsymbol{\theta_c}) = \prod_{\mathbf{x} \in D_c} P(\mathbf{x} \mid \boldsymbol{\theta_c}) \qquad (7.9)$$

  - Applying the MLE to $\boldsymbol{\theta}_c$ is about finding a parameter value $\hat{\theta}_c$ that maximizes the likelihood $P(D_c \mid \theta_c)$. Intuitively, the MLE aims to find a value of θ_c that maximizes the "likelihood" that the data will present.

- Since the product of a sequence in (7.9) can easily lead to underflow, we often use the log-likelihood instead:

$$\begin{aligned} LL(\boldsymbol{\theta_c}) &= \log P(D_c \mid \boldsymbol{\theta_c}) \\ &= \sum_{\mathbf{x} \in D_c} \log P(\mathbf{x} \mid \boldsymbol{\theta}_c) \qquad (7.10) \\ &= -\frac{n_c}{2} \log 2\pi\sigma_c^2 - \Sigma_i \frac{(x_i - \mu_c)^2}{2\sigma_c^2} \end{aligned}$$

with n_c the number of examples in D_c

Maximum Likelihood Estimation

- Let D_c denote the set of class c samples in the training set D, and further suppose the samples are i.i.d. samples. Then, the likelihood of D_c for a given parameter $\boldsymbol{\theta}_c$ is:

$$P(D_c \mid \boldsymbol{\theta_c}) = \prod_{\mathbf{x} \in D_c} P(\mathbf{x} \mid \boldsymbol{\theta_c}) \qquad (7.9)$$

  - Applying the MLE to $\boldsymbol{\theta}_c$ is about finding a parameter value $\hat{\theta}_c$ that maximizes the likelihood $P(D_c \mid \theta_c)$. Intuitively, the MLE aims to find a value of θ_c that maximizes the "likelihood" that the data will present.

- Since the product of a sequence in (7.9) can easily lead to underflow, we often use the log-likelihood instead:

$$\begin{aligned} LL(\boldsymbol{\theta_c}) &= \log P(D_c \mid \boldsymbol{\theta_c}) \\ &= \sum_{\mathbf{x} \in D_c} \log P(\mathbf{x} \mid \boldsymbol{\theta}_c) \qquad (7.10) \end{aligned}$$

- and the MLE of $\boldsymbol{\theta}_c$ is $\hat{\boldsymbol{\theta}}_c$:

$$\hat{\boldsymbol{\theta}}_{\boldsymbol{c}} = \underset{\boldsymbol{\theta}_c}{\operatorname{argmax}} LL(\boldsymbol{\theta}_c) \quad (7.11)$$

Maximum Likelihood Estimation

- For example, suppose the features are continuous and the probability density function follows the Gaussian distribution $p(\mathbf{x} \mid c) \sim N(\boldsymbol{\mu}_c, \boldsymbol{\sigma}_c^2)$, then the MLE of the parameters $\boldsymbol{\mu}_c$ and $\boldsymbol{\sigma}_c^2$ are

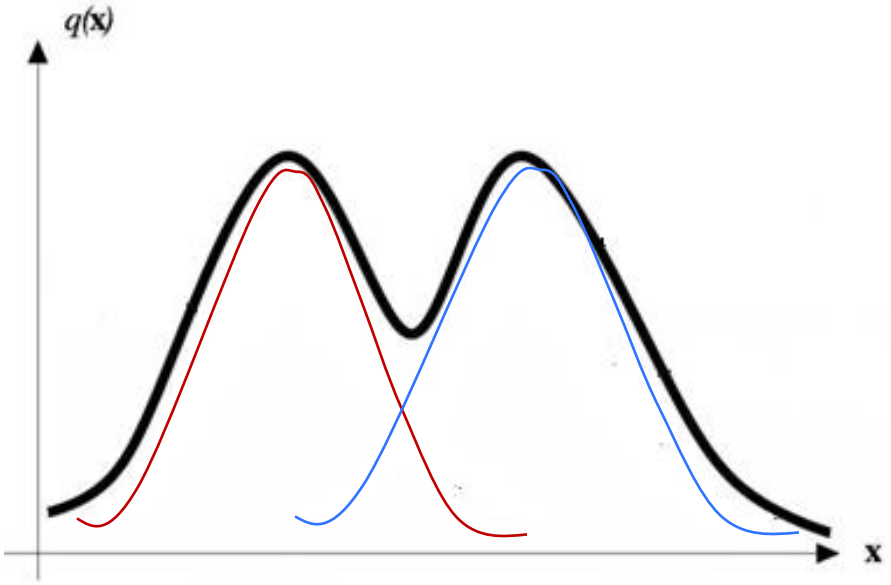
$$\hat{\boldsymbol{\mu}}_c = \frac{1}{|D_c|} \sum_{\mathbf{x} \in D_c} \mathbf{x} \qquad (7.12)$$

$$\hat{\boldsymbol{\sigma}}_c^2 = \frac{1}{|D_c|} \sum_{\mathbf{x} \in D_c} (\mathbf{x} - \hat{\boldsymbol{\mu}}_c)(\mathbf{x} - \hat{\boldsymbol{\mu}}_c)^{\mathrm{T}} \qquad (7.13)$$

- In other words, the estimated mean of Gaussian distribution obtained by the MLE is the sample mean, and the estimated variance is the mean of $(\mathbf{x} - \hat{\boldsymbol{\mu}}_c)(\mathbf{x} - \hat{\boldsymbol{\mu}}_c)^{\mathrm{T}}$; this conforms to our intuition.

Maximum Likelihood Estimation

- Such kind of parametric methods simplify the estimation of posterior probabilities, but the accuracy of estimation heavily relies on whether the hypothetical probability distribution matches the unknown ground-truth data distribution. In practice, a "guessed" probability distribution could lead to misleading results.



Chapter List

- Bayes Decision Theory
- Maximum Likelihood Estimation
- **Naïve Bayes Classifier**
- Semi-Naïve Bayes Classifier
- Bayesian Network
- EM Algorithm

Naïve Bayes Classifier

- The difficulty of estimating the posterior probability $P(c \mid \mathbf{x})$: it is not easy to calculate the class-conditional probability $P(\mathbf{x} \mid c)$ from the finite training samples since $P(\mathbf{x} \mid c)$ is the joint probability on all attributes.
  - For example, d binary properties $\rightarrow$ 2^d possible values, $2^d >>$ (the number of samples) m

In Watermelon data we have 3*3*3*3*3*2=486 combinations of values and 17 examples

ID	color	root	sound	texture	umbilicus	surface	ripe (Class)
1	green	curly	muffled	clear	hollow	hard	TRUE
2	dark	curly	dull	clear	hollow	hard	TRUE
3	dark	curly	muffled	clear	hollow	hard	TRUE
4	green	curly	dull	clear	hollow	hard	TRUE
5	light	curly	muffled	clear	hollow	hard	TRUE
6	green	slightly curly	muffled	clear	slightly hollow	soft	TRUE
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	TRUE
8	dark	slightly curly	muffled	clear	slightly hollow	hard	TRUE
9	dark	slightly curly	dull	slightly blurry	slightly hollow	hard	FALSE
10	green	straight	crisp	clear	flat	soft	FALSE
11	light	straight	dull	blurry	flat	hard	FALSE
12	light	curly	muffled	blurry	flat	soft	FALSE
13	green	slightly curly	muffled	slightly blurry	hollow	hard	FALSE
14	light	slightly curly	dull	slightly blurry	hollow	hard	FALSE
15	dark	slightly curly	muffled	clear	slightly hollow	soft	FALSE
16	light	curly	muffled	blurry	flat	hard	FALSE
17	green	curly	dull	slightly blurry	slightly hollow	hard	FALSE

Naïve Bayes Classifier

- The difficulty of estimating the posterior probability $P(c \mid \mathbf{x})$: it is not easy to calculate the class-conditional probability $P(\mathbf{x} \mid c)$ from the finite training samples since $P(\mathbf{x} \mid c)$ is the joint probability on all attributes.
  - For example, d binary properties $\rightarrow$ 2^d possible values, $2^d >>$ (the number of samples) m

- To avoid this, the naïve Bayes classifier makes the "attribute conditional independence assumption": given any known class, assume all attributes are independent of each other.
- With the attribute conditional independence assumption, we rewrite (7.8) as:

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})} = \frac{P(c)}{P(\mathbf{x})} \prod_{i=1}^{d} P(x_i \mid c) \qquad (7.14)$$

  - where d is the number of attributes and x_i is the value taken on the ith attribute of $\mathbf{x}$

Naïve Bayes Classifier

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})} = \frac{P(c)}{P(\mathbf{x})} \prod_{i=1}^{d} P(x_i \mid c) \qquad (7.14)$$

Naïve Bayes Classifier

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})} = \frac{P(c)}{P(\mathbf{x})} \prod_{i=1}^{d} P(x_i \mid c) \qquad (7.14)$$

Since $P(x)$ is the same for all classes, from the Bayes decision rule (7.6), we have

$$h_{nb}(\mathbf{x}) = \underset{c \in y}{\operatorname{argmax}} P(c) \prod_{i=1}^{d} P(x_i \mid c) \qquad (7.15)$$

- which is the formulation of the naïve Bayes classifier.

Naïve Bayes Classifier

- To train a naïve Bayes classifier, we compute the prior probability $P(c)$ from the training set D and then compute the conditional probability $P(x_i \mid c)$ for each attribute.
  - Let D_c denote a subset of D containing all samples of class c. Then, given sufficient i.i.d. samples, the prior probability can be easily estimated by

$$P(c) = \frac{|D_c|}{D} \qquad (7.16)$$

  - For discrete attributes, let D_{c,x_i} denote a subset of D_c containing all samples taking the value x_i on the i-th attribute. Then, the conditional probability $P(x_i \mid c)$ can be estimated by

$$P(x_i \mid c) = \frac{|D_{c,x_i}|}{D} \qquad (7.17)$$

  - For continuous features, suppose $p(x_i \mid c) \sim N(\mu_{c,i}, \sigma_{c,i}^2)$, where $\mu_{c,i}$ and $\sigma_{c,i}^2$ are, respectively, the mean and variance of the i-th feature of class c samples. Then, we have

$$P(x_i \mid c) = \frac{1}{\sqrt{2\pi}\sigma_{c,i}} \exp\left(-\frac{(x_i - \mu_{c,i})^2}{2{\sigma_{c,i}}^2}\right) \qquad (7.18)$$

Naïve Bayes Classifier

- Now, let us train a naïve Bayes classifier using the watermelon data set 3.0 classify the following watermelon T1:

ID	color	root	sound	texture	umbilicus	surface	density	sugar	ripe
T1	green	curly	muffled	clear	hollow	hard	0.697	0.460	?

Laplacian Correction

- If a feature value has never appeared together with a particular class, it becomes problematic to use the probability. For instance, given a testing sample with *sound* = *crisp*, the naïve Bayes classifier trained on the Watermelon data set 3.0 will predict 0.
- The classification result will always be *ripe* = *false* regardless of the values of other features.
- Such a behavior is unreasonable.

Laplacian Correction

- To avoid "removing" the information carried by other features, a common choice is the Laplacian correction.
  - let N denote the number of distinct classes in the training set D, N_i denote the number of distinct values the ith feature can take. Then, (7.16) and (7.17) can be, respectively, corrected as

$$\hat{P}(c) = \frac{|D_c| + 1}{|D| + N} \quad (7.19) \qquad \hat{P}(x_i \mid c) = \frac{|D_{c,x_i}| + 1}{|D| + N_i} \quad (7.20)$$

- There are different ways of using a naïve Bayes classifier. If the speed of prediction is important, then a naïve Bayes classifier can pre-calculate all relevant probabilities and save for later use; If the training data changes frequently, then we can take a lazy learning approach.

Chapter List

- Bayes Decision Theory
- Maximum Likelihood Estimation
- Naïve Bayes Classifier
- **Semi-Naïve Bayes Classifier**
- Bayesian Network
- EM Algorithm

Semi-Naïve Bayes Classifier

- To overcome the difficulty of computing $P(c \mid \mathbf{x})$ in Bayes' theorem, the naïve Bayes classifier makes the attribute conditional independence assumption, which often does not hold in practice. Hence, semi-naïve Bayes classifiers are developed to relax this assumption to some extent.

Semi-Naïve Bayes Classifier

- A general strategy used by semi-naïve Bayes classifiers is *One-Dependent Estimator* (ODE). *one-dependent* means each feature can depend on at most one feature other than the class information, that is,

$$P(c \mid x) \propto P(c) \prod_{i=1}^{d} P(x_i \mid c, pa_i)$$

  - where x_i depends on pa_i, and pa_i is called the parent feature of x_i

- For each feature x_i, if the parent feature pa_i is known, then $P(x_i \mid c, pa_i)$ can be estimated.

Hence, the key problem becomes determining the parent features.

SPODE

- The most straightforward approach is called Super-Parent ODE (SPODE), which assumes that all features depend on just one feature called the *super-parent*. SPODE selects the super-parent feature using model selection methods such as cross-validation.

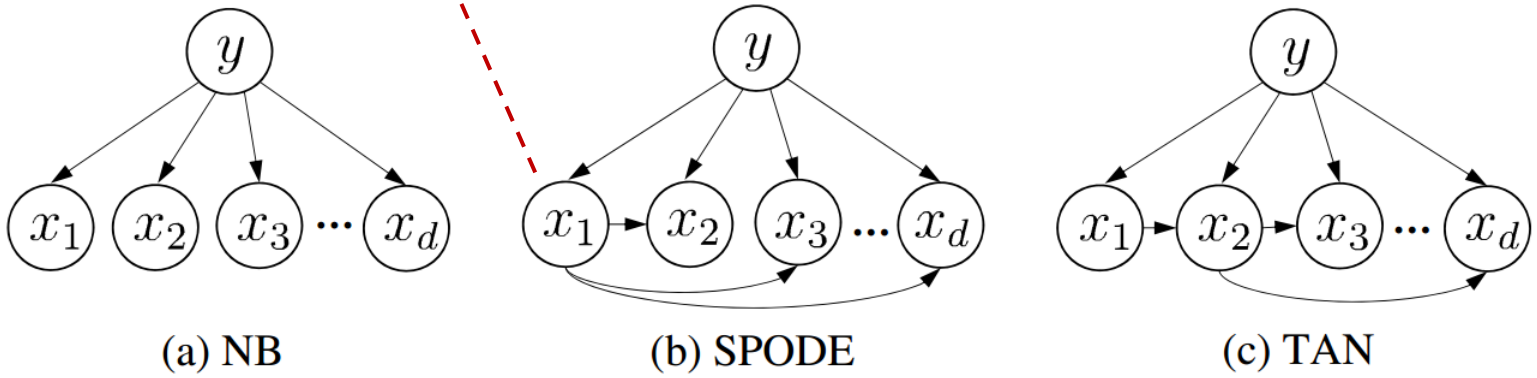


Fig. 7.1: Feature dependencies of naïve Bayes and semi-naïve Bayes classifiers.

- In Figure 7.1(b), x_1 **is a super-parent feature.**

TAN

- TAN (Tree augmented Naïve Bayes) [Friedman et al., 1997] , which is based on maximum weighted spanning tree [Chow and Liu, 1968], simplifies feature dependencies into a tree structure, as shown in Figure 7.1(c):
  - Compute the conditional mutual information for each pair of features:

$$I(x_i, x_j \mid y) = \sum_{x_i, x_j; c \in y} P(x_i, x_j \mid c) \log \frac{P(x_i, x_j \mid c)}{P(x_i \mid c) P(x_j \mid c)}$$

  - Construct a complete undirected graph in which the nodes are features. Set $I(x_i, x_j \mid y)$ as the weight;
  - Construct a maximum weighted spanning tree and select a root feature. Set the direction of each edge outward from the root feature;
  - Add a class node y and add direct edges from y to other feature nodes.

On Minimum Weight Spanning Trees

A note

- Are useful in **causal discovery** (to simply complex graphs and highlight the root-causes in process disturbances) or in cost-effective **network design**
- A **Minimum Spanning Tree** (MST) or minimum weight spanning tree is:
  1. a subset of the edges of a connected, edge-weighted undirected graph
  2. it connects all the vertices together, without any cycles
  3. with the minimum possible total edge weight.

  It is a spanning tree whose sum of edge weights is as small as possible.
- To convert MST into a Maximum Weight Spanning tree, multiply weights by -1 and then use an algorithm for MST:
  - **Kruskal's Algorithm:** Sort the edges by their (negated) weights in ascending order. Add edges to the growing tree as long as they do not form a cycle.
  - **Prim's Algorithm:** Start from an arbitrary vertex and iteratively add the highest-weight (negated) edge connecting a vertex in the tree to a vertex outside the tree, ensuring no cycles are formed.

AODE

- Averaged One-Dependent Estimator (AODE) [Webb et al., 2005] is a more powerful one-dependent classifier, which takes advantage of ensemble learning.
  - Try to use each feature as a super-parent to build multiple SPODE models
  - Integrate those supported by sufficient training data:

$$P(c \mid \mathbf{x}) \propto \sum_{i=1;|\mathrm{D_{x_i}}| \geq \mathrm{m}'}^{d} P(c, x_i) \prod_{j=1}^{d} P(x_j \mid c, x_i)$$

where D_{x_i} is the subset of samples taking the value x_i on the i-th feature, and m' is a threshold constant.

$$\hat{P}(c, x_i) = \frac{|D_{c,x_i}| + 1}{|D| + N \times N_i} \qquad \hat{P}(x_j \mid c, x_i) = \frac{|D_{c,x_i,x_j}| + 1}{|D_{c,x_i}| + N_j}$$

where N is the number of distinct classes in D, N_i is the number of distinct values the i-th feature can take, D_{c,x_i} is the subset of class c samples taking the value x_i on the i th feature

Chapter List

- Bayes Decision Theory
- Maximum Likelihood Estimation
- Naïve Bayes Classifier
- Semi-Naïve Bayes Classifier
- **Bayesian Network**
- EM Algorithm

Bayesian Network

- *Bayesian networks*, also known as *belief networks*, utilize Directed Acyclic Graphs (DAG) to represent dependencies among features and build Conditional Probability Tables (CPT) to describe the joint probabilities of features.

Bayesian Network

- *Bayesian networks*, also known as *belief networks*, utilize Directed Acyclic Graphs (DAG) to represent dependencies among features and build Conditional Probability Tables (CPT) to describe the joint probabilities of features.

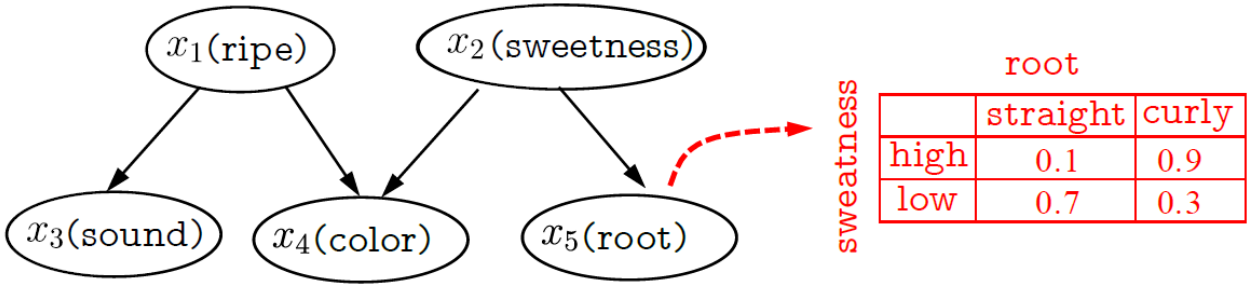


Fig. 7.2: The Bayesian network structure of the watermelon problem together with the conditional probability table of *root*.

- From the network structure, we see that color directly depends on ripe and sweetness.
- From the CPT, we can further obtain a quantified dependency of root on sweetness, e.g.,

$$P(\text{root} = \text{straight} \mid \text{sweetness} = \text{high}) = 0.1.$$

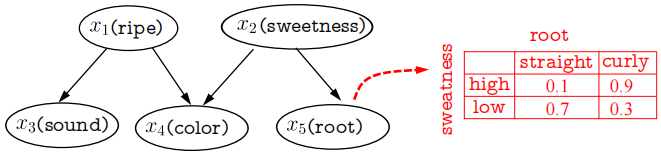
Bayesian Network:
Network Structure

- The Bayesian network structure effectively expresses the conditional independence among features. Given the sets of parent nodes, the **Bayesian network assumes each feature is independent of its all non-descendant features**.
- $B = \langle G, \Theta \rangle$ defines the joint probability of the features $x_1, x_2, \cdots, x_d$

$$P(x_1, x_2, x_3, x_4, x_5) = P(x_1)P(x_2)P(x_3 \mid x_1)P(x_4 \mid x_1, x_2)P(x_5 \mid x_2) \quad (7.26)$$

Taking Figure 7.2 as an example, the joint probability is de

$$P_B(x_1, x_2, \cdots, x_d) = \prod_{i=1}^{d} P_B(x_i \mid \pi_i) = \prod_{i=1}^{d} \theta_{x_i|\pi_i}$$



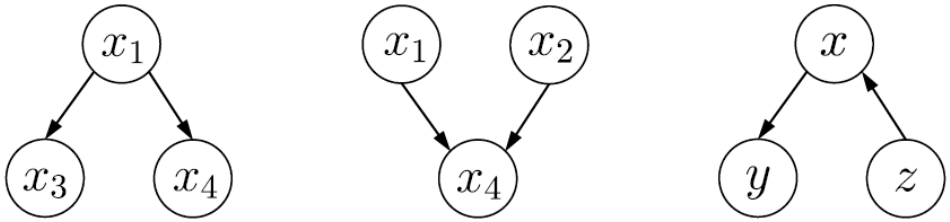
We can see $x_3 \perp x_4 \mid x_1$ and x_3 are independent given the value of x_1 and x_4 (i.e.,).

d-separated

parents of feature x_i

Bayesian Network:
Network Structure

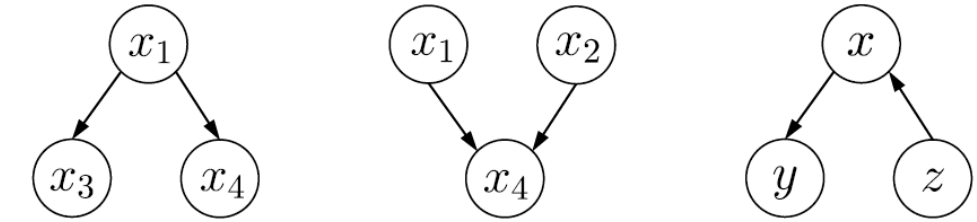
- The following shows three typical cases of dependencies among three variables:



(a) Common parent. (b) V-structure. (c) Cascade.

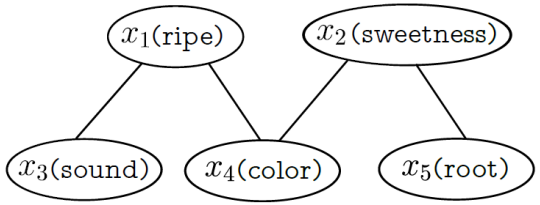
Bayesian Network:
Network Structure

- The following shows three typical cases of dependencies among three variables:



(a) Common parent. (b) V-structure. (c) Cascade.

- We can use d-separation to analyze the conditional independence among variables in a directed graph.
  - Find all V-structures and then add an undirected edge
  - Change all directed edges to undirected edges.



The undirected graph produced by this process is called a *moral graph*.

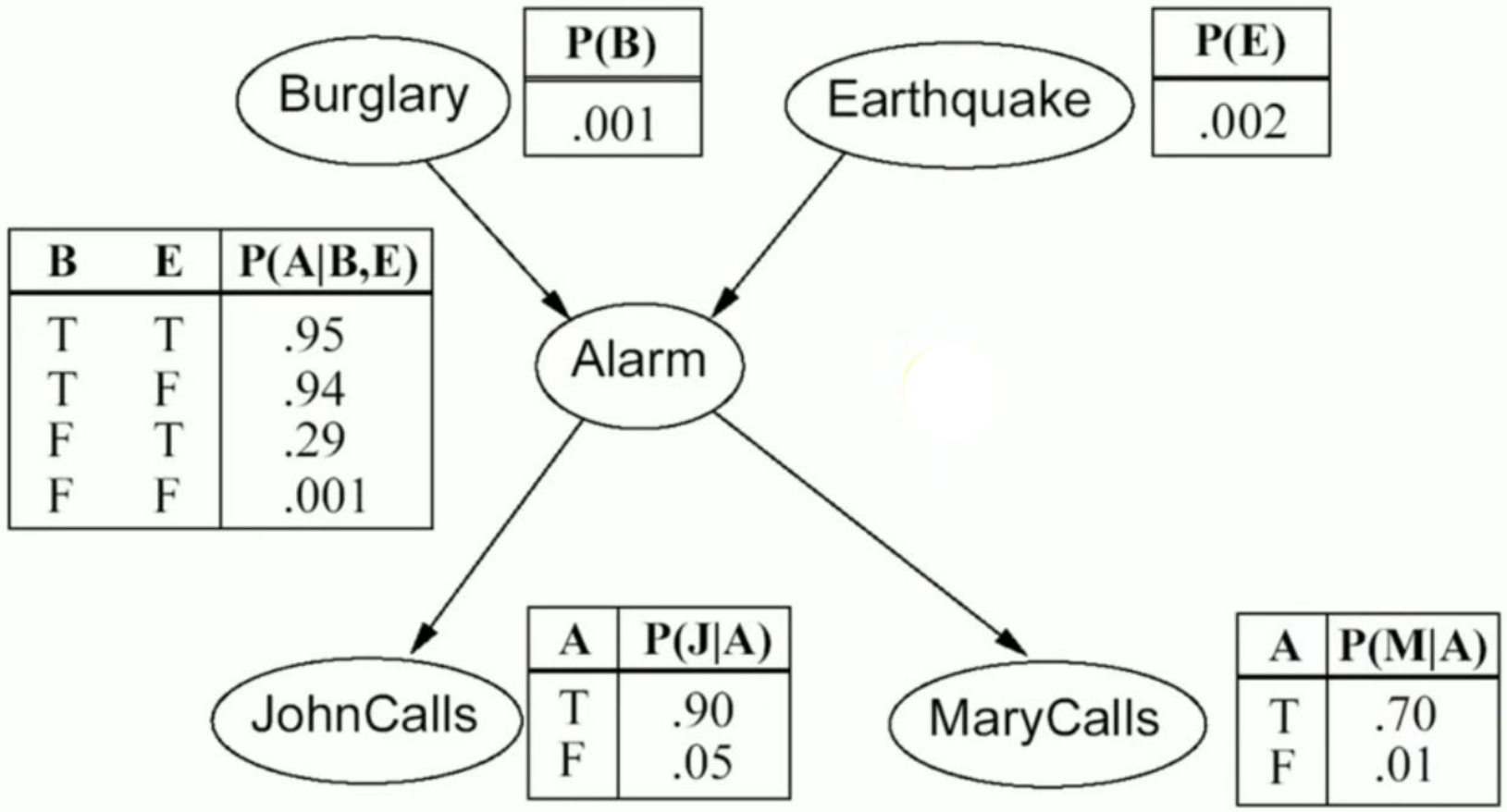
The name stems from the fact that, **in a moral graph, two nodes that have a common child are required to be married by sharing an edge**.

Bayesian Network:
Inference

- The process of inferring the value of a *queried feature* from the observed feature values is called *inference*, where the observed feature values are called *evidence*.
- Ideally, we wish to use the joint probability distribution defined by the Bayesian network to compute the exact posterior probabilities of the queried feature given the evidence.

Some well-known examples of BN (see also https://www.bnlearn.com/)

- **Alarm**



Some well-known examples of BN (see also https://www.bnlearn.com/)

Onset type -2 Diabetes

		Diabetes	
		Y	N
Waist Circ.	Y	5.88%	94.12 %
	N	13.20 %	86.80 %

Waist circumference (<90 cm in men; <80 in women)

		Diabetes	
		Y	N
Hyper-tension	Y	12.55 %	87.45 %
	N	5.39 %	94.61 %

Hypertension

fasting plasma glucose (FPG) levels of ≥ 5.6

		Diabetes	
		Y	N
FPG	Y	17.36 %	82.64 %
	N	4.33%	95.67 %

Diabetes

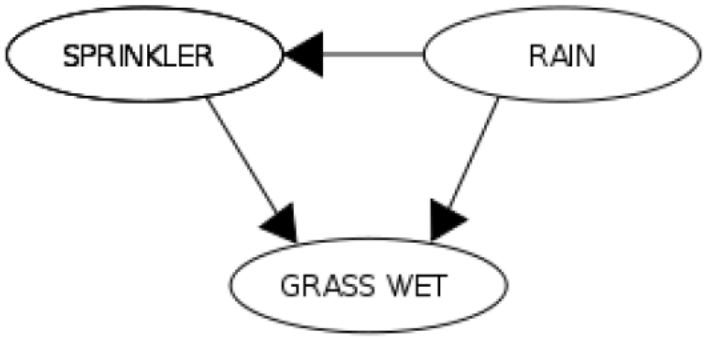
Diabetes	
YES	8.17 %
NO	91.83 %

Wang Y, Zhang WS, Hao YT, Jiang CQ, Jin YL, Cheng KK, Lam TH and Xu L (2022) A Bayesian network model of new-onset diabetes in older Chinese: The Guangzhou biobank cohort study. Frontiers Endocrinol. 13:916851. doi: 10.3389/fendo.2022.916851

Some well-known examples (see also https://www.bnlearn.com/)

- **Wet grass**

RAIN	SPRINKLER T	SPRINKLER F
F	0.4	0.6
T	0.01	0.99



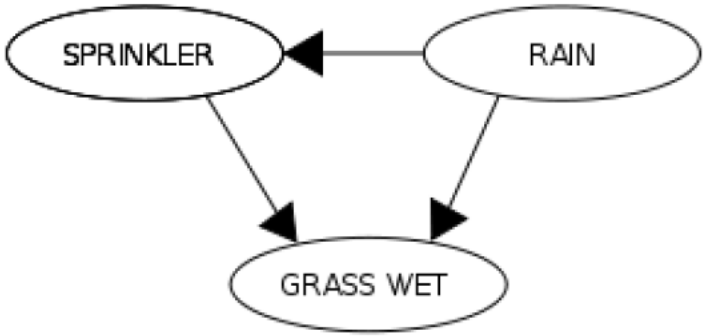
RAIN T	RAIN F
0.2	0.8

SPRINKLER	RAIN	GRASS WET T	GRASS WET F
F	F	0.0	1.0
F	T	0.8	0.2
T	F	0.9	0.1
T	T	0.99	0.01

Exercize on Inference using a BN

- **Wet grass**

RAIN	SPRINKLER T	SPRINKLER F
F	0.4	0.6
T	0.01	0.99



RAIN T	RAIN F
0.2	0.8

- **Question:**

- Given that you observe wet grass (evidence), which is the probability that it rained (queried feature)?

SPRINKLER	RAIN	GRASS WET T	GRASS WET F
F	F	0.0	1.0
F	T	0.8	0.2
T	F	0.9	0.1
T	T	0.99	0.01

Recall: Fundamental rules of probabilities

- **Unitary sum of the probabilities of a random variable over the domain of the random variable**

$$1 = \Sigma_{y_i \in Dom(Y)} P(Y = y_i)$$

- **Sum rule (marginalization)**

$$p(X = x_i) = \Sigma_{y_i \in Dom(Y)} p(X = x_i, Y = y_i)$$

- **Product rule**

$$P(Y|X) = \frac{P(X,Y)}{P(X)} \Rightarrow P(X,Y) = P(Y|X)P(X)$$

- **Independent variables**

$$P(X,Y) = P(X)P(Y)$$

Marginalisation on: Watermelon 2.0

ID	color	root	sound	texture	umbilicus	surface	ripe (Class)
1	green	curly	muffled	clear	hollow	hard	TRUE
2	dark	curly	dull	clear	hollow	hard	TRUE
3	dark	curly	muffled	clear	hollow	hard	TRUE
4	green	curly	dull	clear	hollow	hard	TRUE
5	light	curly	muffled	clear	hollow	hard	TRUE
6	green	slightly curly	muffled	clear	slightly hollow	soft	TRUE
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	TRUE
8	dark	slightly curly	muffled	clear	slightly hollow	hard	TRUE
9	dark	slightly curly	dull	slightly blurry	slightly hollow	hard	FALSE
10	green	straight	crisp	clear	flat	soft	FALSE
11	light	straight	dull	blurry	flat	hard	FALSE
12	light	curly	muffled	blurry	flat	soft	FALSE
13	green	slightly curly	muffled	slightly blurry	hollow	hard	FALSE
14	light	slightly curly	dull	slightly blurry	hollow	hard	FALSE
15	dark	slightly curly	muffled	clear	slightly hollow	soft	FALSE
16	light	curly	muffled	blurry	flat	hard	FALSE
17	green	curly	dull	slightly blurry	slightly hollow	hard	FALSE

Marginalisation on Watermelon 2.0, on *color* and *root*:

ID	color	root
1	green	curly
2	dark	curly
3	dark	curly
4	green	curly
5	light	curly
6	green	slightly curly
7	dark	slightly curly
8	dark	slightly curly
9	dark	slightly curly
10	green	straight
11	light	straight
12	light	curly
13	green	slightly curly
14	light	slightly curly
15	dark	slightly curly
16	light	curly
17	green	curly

⇒

Contingency table

		ROOT			
		curly	slightly curly	straight	SUM
COLOR	green	3	2	1	**6**
	dark	2	4	0	**6**
	light	3	1	1	**5**
	SUM	**8**	**7**	**2**	**17**

Probability Table:

$P(Color = green)$

$P(Color = green, Root = curly)$

		ROOT			
		curly	slightly curly	straight	SUM
COLOR	green	3/17	2/17	1/17	**6/17**
	dark	2/17	4/17	0	**6/17**
	light	3/17	1/17	1/17	**5/17**
	SUM	**8/17**	**7/17**	**2/17**	**17/17**

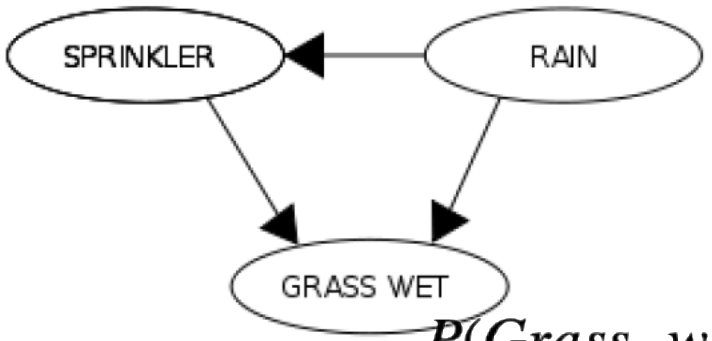
$P(Root = curly)$

Exercize:

- **Wet grass**

$P(Sprinkle \mid Rain)$

RAIN	SPRINKLER T	SPRINKLER F
F	0.4	0.6
T	0.01	0.99



RAIN T	RAIN F
0.2	0.8

$P(Rain)$

- **Question:**

$P(Grass_wet \mid Sprinkle, Rain)$

SPRINKLER	RAIN	GRASS WET T	GRASS WET F
F	F	0.0	1.0
F	T	0.8	0.2
T	F	0.9	0.1
T	T	0.99	0.01

- Given that you observed grass wet, which is the probability that it rained?

$$P(Rain = T \mid Grass_wet = T) = \frac{P(Rain = T, Grass_wet = T)}{P(Grass_wet = T)}$$

$$P(Rain = T, Grass_wet = T) = \Sigma_{Sprinkle} P(Rain = T, Grass_wet = T, Sprinkle)$$

$$= \Sigma_{Sprinkle} P(Grass_wet = T \mid Rain = T, Sprinkle) P(Rain = T, Sprinkle)$$

?

P(Sprinkle,Rain=T)

$P(Sprinkle \mid Rain)$

RAIN	SPRINKLER T	SPRINKLER F
F	0.4	0.6
T	0.01	0.99

$P(Rain)$

RAIN T	RAIN F
0.2	0.8

$$P(Sprinkle, Rain) = P(Sprinkle \mid Rain)P(Rain)$$

$$\Downarrow$$

$$P(Sprinkle, Rain = T) = P(Sprinkle \mid Rain = T)P(Rain = T)$$

$$= P(Sprinkle = T \mid Rain = T)P(Rain = T) + P(Sprinkle = N \mid Rain = T)P(Rain = T)$$

$$= 0.01 * 0.2 + 0.99 * 0.2 = 0.2 = P(Rain = T)$$

$$P(Sprinkle,Rain=F) = P(Sprinkle=T|Rain=F)P(Rain=F) + P(Sprinkle=F|Rain=F)P(Rain=F) =$$
$$=0.4*0.8+0.6*0.8=0.8=P(Rain=F)$$

⇒ We discovered that *Sprinkle* and *Rain* are independent variables

P(Rain=T,Grass_wet=T)

Numerator of the amount we want to compute

$$P(Grass_wet \mid Sprinkle, Rain)$$

		GRASS WET	
SPRINKLER	RAIN	T	F
F	F	0.0	1.0
F	T	0.8	0.2
T	F	0.9	0.1
T	T	0.99	0.01

$$P(Rain = T, Grass_wet = T) = \Sigma_{Sprinkle} P(Rain = T, Grass_wet = T, Sprinkle)$$

$$= \Sigma_{Sprinkle} P(Grass_wet = T \mid Rain = T, Sprinkle) P(Rain = T, Sprinkle)$$

$$= 0.8 * 0.2 + 0.99 * 0.2 = 0.358$$

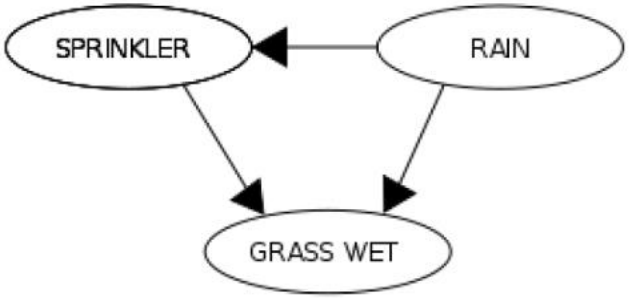
$P(Sprinkle, Rain = T)$

$$P(Rain = T, Grass_wet = T) = 0.358$$

(numerator of the amount we want to obtain)

P(Grass_wet)

Denominator of the amount we want



SPRINKLER	RAIN	GRASS WET T	GRASS WET F
F	F	0.0	1.0
F	T	0.8	0.2
T	F	0.9	0.1
T	T	0.99	0.01

$$P(Grass_wet = T) =$$

$$P(Grass_wet = T \mid Sprinkler = F, Rain = T)P(Sprinkler = F, Rain = T)$$

$$+P(Grass_wet = T \mid Sprinkler = T, Rain = F)P(Sprinkler = T, Rain = F)$$

$$+P(Grass_wet = T \mid Sprinkler = T, Rain = T)P(Sprinkler = T, Rain = T)$$

$$= 0.8 \cdot P(Sprinkler = F, Rain = T) + 0.9 \cdot P(Sprinkler = T, Rain = F) + 0.99 \cdot P(Sprinkler = T, Rain = T)$$

P(Sprinkle, Rain)

Factors for obtaining P(Grass_wet)

$P(Sprinkle \mid Rain)$

RAIN	SPRINKLER T	SPRINKLER F
F	0.4	0.6
T	0.01	0.99

$P(Rain)$

RAIN T	RAIN F
0.2	0.8

$$P(Sprinkle = F, Rain = T) = P(Sprinkle = F \mid Rain = T)P(Rain = T)$$

$$P(Sprinkle = F, Rain = T) = 0.99 \cdot 0.2 = 0.198$$

$$P(Sprinkle = T, Rain = F) = P(Sprinkle = T \mid Rain = F)P(Rain = F)$$

$$P(Sprinkle = T, Rain = F) = 0.4 \cdot 0.8 = 0.32$$

$$P(Sprinkle = T, Rain = T) = P(Sprinkle = T \mid Rain = T)P(Rain = T)$$

$$P(Sprinkle = T, Rain = T) = 0.01 \cdot 0.2 = 0.002$$

P(Grass_wet)

Denominator of the amount we want

$$P(Grass_wet = T) =$$

$$= 0.8 \cdot P(Sprinkler = F, Rain = T) + 0.9 \cdot P(Sprinkler = T, Rain = F) + 0.99 \cdot P(Sprinkler = T, Rain = T)$$

We substitute the obtained terms:

$$P(Sprinkle = F, Rain = T) = 0.198$$

$$P(Sprinkle = T, Rain = F) = 0.32$$

$$P(Sprinkle = T, Rain = T) = 0.002$$

$$= 0.8 \cdot 0.198 + 0.9 \cdot 0.32 + 0.99 \cdot 0.002 = 0.448$$

Final computation

- **Question:**

- Given that you observed grass wet, which is the probability that it rained?

$$P(Rain = T \mid Grass_wet = T) = \frac{P(Rain = T, Grass_wet = T)}{P(Grass_wet = T)}$$

$$= \frac{0.358}{0.448} = 0.799$$

Bayesian Network:
Learning

- The primary task of Bayesian network learning is to find the most "appropriate" Bayesian network
- We first define a score function to evaluate the fitness between a Bayesian network and a training set
  - According to the Minimum Description Length (MDL) principle, we search for the Bayesian network with the shortest combined code length.

Introduction to Minimum Description Length (MDL)

From Wikipedia:

- MDL has its origins in Information Theory and has been developed within the fields of statistics, theoretical computer science and machine learning, and more narrowly Computational Learning Theory.
- Historically, there are meanings of "*the* minimum description length *principle*" that vary in what is meant by *description*
- Within Rissanen's theory of learning, models are statistical hypotheses and descriptions are defined as universal codes.
- Models of a given set of observations, encoded as bits of data, could be created in the form of computer programs that output that data
- Rissanen was the first to automatically derive short descriptions (Bayesian Information Criterion - BIC).
- Within Algorithmic Information Theory, the description length of a data sequence is the **length of the smallest program that outputs that data set**.
  It is closely related to Solomonoff's theory of inductive inference: the best model of a data set is represented by its shortest self-extracting archive.

★ Compression-based models I

(Slides taken from Peter Flach's book on ML)

Consider the maximum a posteriori decision rule again:

$$y_{\text{MAP}} = \arg\max_y P(X = x|Y = y)P(Y = y)$$

Taking negative logarithms, we can turn this into an equivalent minimisation:

$$y_{\text{MAP}} = \arg\min_y -\log P(X = x|Y = y) - \log P(Y = y)$$

- If an event has probability p of happening, the negative logarithm of p quantifies the *information content* of the message that the event has indeed happened.
- We write $IC(X|Y) = -\log_2 P(X|Y)$ and $IC(Y) = -\log_2 P(Y)$.

$$y_{MAP} = argmin_y\{IC(X = x|Y = y) + IC(Y = y)\}$$

⋆ Compression-based models II

- The unit of information depends on the base of the logarithm: it is customary to take logarithms to the base 2, in which case information is measured in bits.
- For example, if you toss a fair coin once and tell me it came up heads, this contains $-\log_2 1/2 = 1$ bit of information; if you roll a fair die once and let me know it came up six, the information content of your message is $-\log_2 1/6 = 2.6$ bits.

Less bits are needed for shorter messages, that carry a lower amount of information content



Definition 9.1, p.294

⋆ Minimum description length principle

Let $L(m)$ denote the length in bits of a description of model m, and let $L(D|m)$ denote the length in bits of a description of data D given model m (both measured by some near-optimal code L). According to the minimum description length principle, the preferred model is the one minimising the description length of model and data given model:

quantifies the complexity of the model

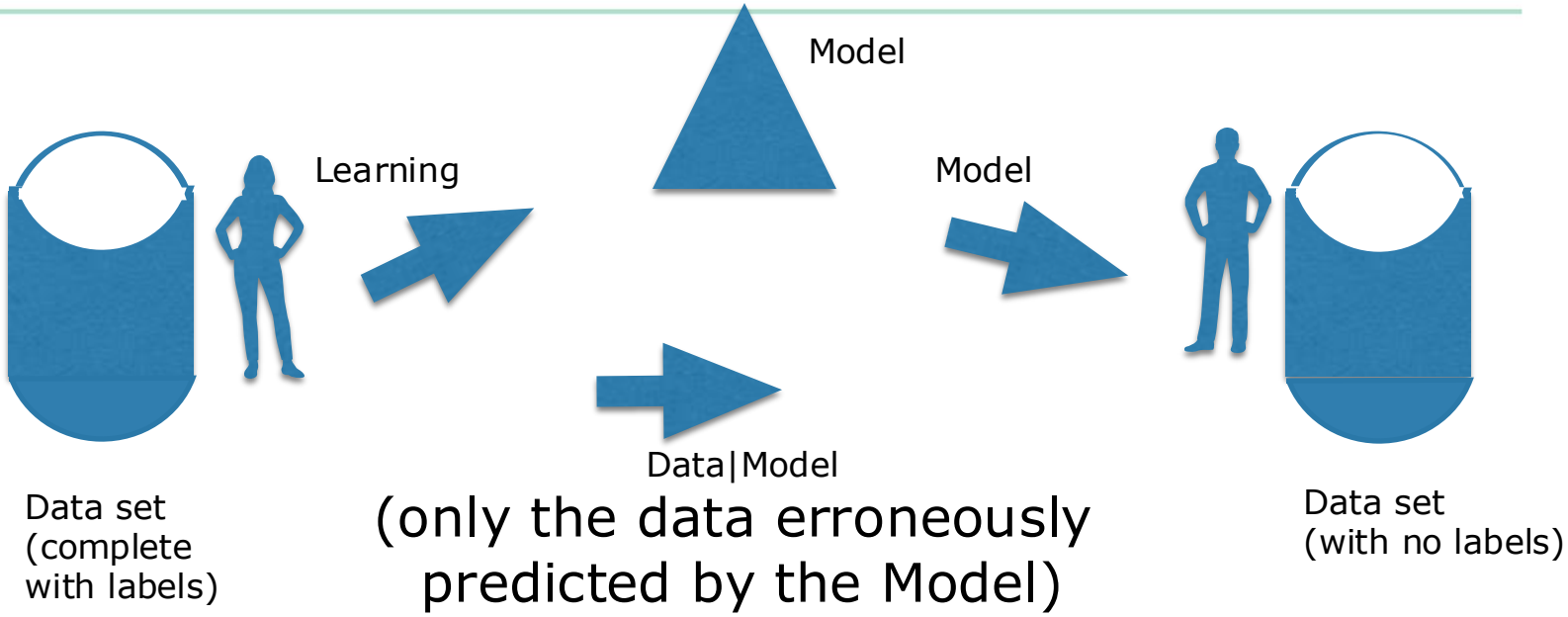
$$m_{\text{MDL}} = \arg\min_{m \in M} \left(L(m) + L(D|m) \right)$$

quantifies the errors of the model

What encoding to use in order to determine the model complexity $L(m)$ is often not straightforward and to some extent subjective. This is similar to the Bayesian perspective, where we need to define a prior distribution on models. The MDL viewpoint offers a concrete way of defining model priors by means of codes.

MDL trades off the complexity and the accuracy of the model

The Minimum Description Length Principle



- The length of the messages sent between the two people measures the cost of the delivery of the information exchanged (quantifies the amount of information of the Model and of the Data).
- $Cost(Model, Data) = Cost(Model) + Cost(Data|Model)$

The Minimum Description Length Principle

- $Cost(Model, Data) = Cost(Model) + Cost(Data|Model)$
- The best model would be that model whose information amount is minimum (minimizes $Cost(Model)$) which is in accordance with Occam's razor principle about the preference towards simple models
- In addition, the best model should also minimise the error of the model on the data which is represented by:
  $Cost(Data|Model)$
  It is the amount of information about the data that we need to represent explicitly, separately by the model.

Bayesian Network:
Learning

- The primary task of Bayesian network learning is to find the most "appropriate" Bayesian network
- We first define a score function to evaluate the fitness between a Bayesian network and a training set
  - According to the Minimum Description Length (MDL) principle, we search for the Bayesian network with the shortest combined code length.
- Given a training set $D = \{x_1, x_2, \cdots, x_m\}$ the score function of a Bayesian network $B = \langle G, \Theta \rangle$ on D can be written as

$$S(B \mid D) = f(\theta)|B| - LL(B \mid D) \qquad (7.28)$$

  - where $|B|$ is the number of parameters of the Bayesian network, $f(\theta)$ is the coding length required to describe the parameter θ, and

$$LL(B \mid D) = \sum_{i=1}^{m} \log P_B(x_i) \qquad (7.29)$$

  the log-likelihood of the Bayesian network.

Bayesian Network:
Approximate Inference

- We said that Bayesian networks are useful because they allow *inference on an interesting feature (queried)*

- Ideally, we wish to use the joint probability distribution defined by the Bayesian network to compute the exact posterior probabilities.
- A typical approximate method for Bayesian networks is the Gibbs sampling.

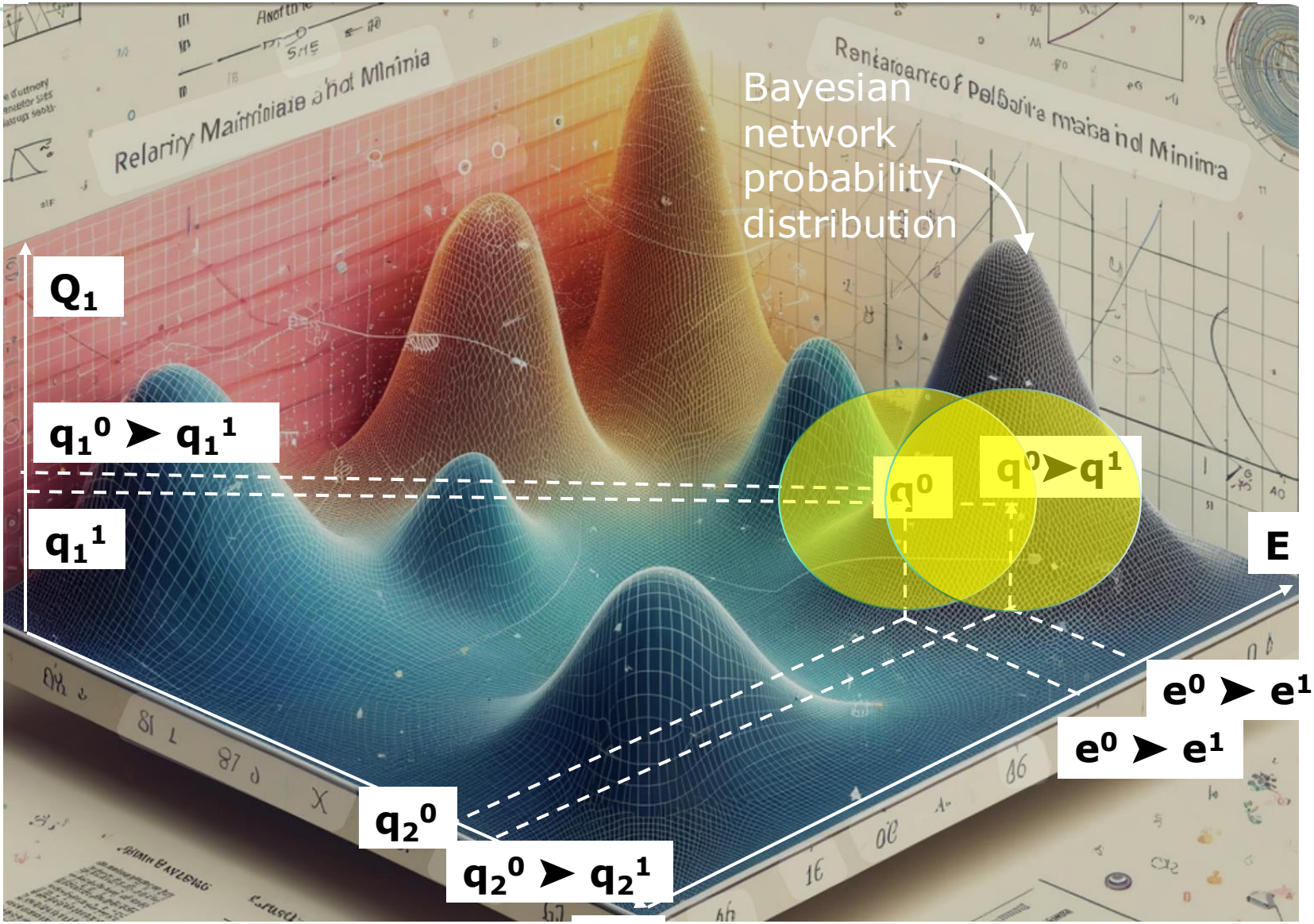
Bayesian Network:
Approximate Inference

- We said that Bayesian networks are useful because they allow *inference on an interesting feature (queried)*
- Ideally, we wish to use the joint probability distribution defined by the Bayesian network to compute the exact posterior probabilities.
- A typical approximate method for Bayesian networks is the Gibbs sampling.
- The Gibbs sampling algorithm starts with a randomly generated sample q^0 , and generates a new sample by modifying the non-evidence variables of the current sample. Suppose we sampled T times and obtained n_q samples that are consistent with the query q , then the posterior probability can be approximated as

$$P(Q = q \mid E = e) \simeq \frac{n_q}{T} \qquad (7.33)$$

- Since every step only depends on the state of the previous step, the sampling generates a *Markov chain*

Animation of Gibbs sampling



Bayesian Network:
Approximate Inference

Algorithm 7.1 Gibbs Sampling.

Input: Bayesian network $B = \langle G, \Theta \rangle$;
Number of samplings T;
Evidence variables $\mathbf{E}$ and their values $\mathbf{e}$;
Query variables $\mathbf{Q}$ and their values $\mathbf{q}$.

Process:

$n_q = 0$;
$\mathbf{q}^0 =$ assign random initial values to $\mathbf{Q}$;
for $t = 1, 2, \ldots, T$ **do**
  for $Q_i \in \mathbf{Q}$ **do**
    $\mathbf{Z} = \mathbf{E} \cup \mathbf{Q} \backslash \{Q_i\}$;
    $\mathbf{z} = \mathbf{e} \cup \mathbf{q}^{t-1} \backslash \{q_i^{t-1}\}$;
    Compute distribution $P_B(Q_i \mid \mathbf{Z} = \mathbf{z})$ according to B;
    $q_i^t = Q_i$ sampled from $P_B(Q_i \mid \mathbf{Z} = \mathbf{z})$;
    $\mathbf{q}^t = \mathbf{q}^{t-1}$ with q_i^{t-1} replaced by q_i^t;
  end for
  if $\mathbf{q}^t = \mathbf{q}$ **then**
    $n_q = n_q + 1$.
  end if
end for

Output: $P(\mathbf{Q} = \mathbf{q} \mid \mathbf{E} = \mathbf{e}) \simeq \frac{n_q}{T}$.

Chapter List

- Bayes Decision Theory
- Maximum Likelihood Estimation
- Naïve Bayes Classifier
- Semi-Naïve Bayes Classifier
- Bayesian Network
- **EM Algorithm**

EM Algorithm

- We often have *incomplete* training samples: the root of a watermelon may have been removed, that is, the feature root is unknown. Then, in the presence of unobserved features, how can we estimate the model parameters?

EM Algorithm

- We often have *incomplete* training samples: the root of a watermelon may have been removed, that is, the feature root is unknown. Then, in the presence of unobserved features, how can we estimate the model parameters?
- Let $\mathbf{X}$ denote the set of observed variables, $\mathbf{Z}$ denote the set of latent variables, and Θ denote the model parameters. Then, the maximum likelihood estimation of Θ maximizes the log-likelihood

$$LL(\boldsymbol{\Theta} \mid \mathbf{X}, \mathbf{Z}) = \ln P(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\Theta}) \qquad (7.34)$$

EM Algorithm

- We often have *incomplete* training samples: the root of a watermelon may have been removed, that is, the feature root is unknown. Then, in the presence of unobserved features, how can we estimate the model parameters?
- Let $\mathbf{X}$ denote the set of observed variables, $\mathbf{Z}$ denote the set of latent variables, and Θ denote the model parameters. Then, the maximum likelihood estimation of Θ maximizes the log-likelihood

$$LL(\boldsymbol{\Theta} \mid \mathbf{X}, \mathbf{Z}) = \ln P(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\Theta}) \qquad (7.34)$$

- We cannot solve (7.34) directly because $\mathbf{Z}$ are latent variables. However, we can use the expectation of $\mathbf{Z}$ to maximize the log marginal likelihood of the observed data:

$$LL(\boldsymbol{\Theta} \mid \mathbf{X}) = \ln P(\mathbf{X} \mid \boldsymbol{\Theta}) = \ln \sum_{\mathbf{Z}} P(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\Theta}) \qquad (7.35)$$

EM Algorithm

The Expectation-Maximization (EM) algorithm [Dempster et al., 1977] is a powerful iterative method for estimating latent variables.

- Given the value of Θ, from the training data, we can infer the expected value for each latent variable in $\mathbf{Z}$ (the E-step))
- Given the values of the latent variables in $\mathbf{Z}$, we can estimate Θ with the maximum likelihood estimation (the M-step).

EM Algorithm

The Expectation-Maximization (EM) algorithm [Dempster et al., 1977] is a powerful iterative method for estimating latent variables.

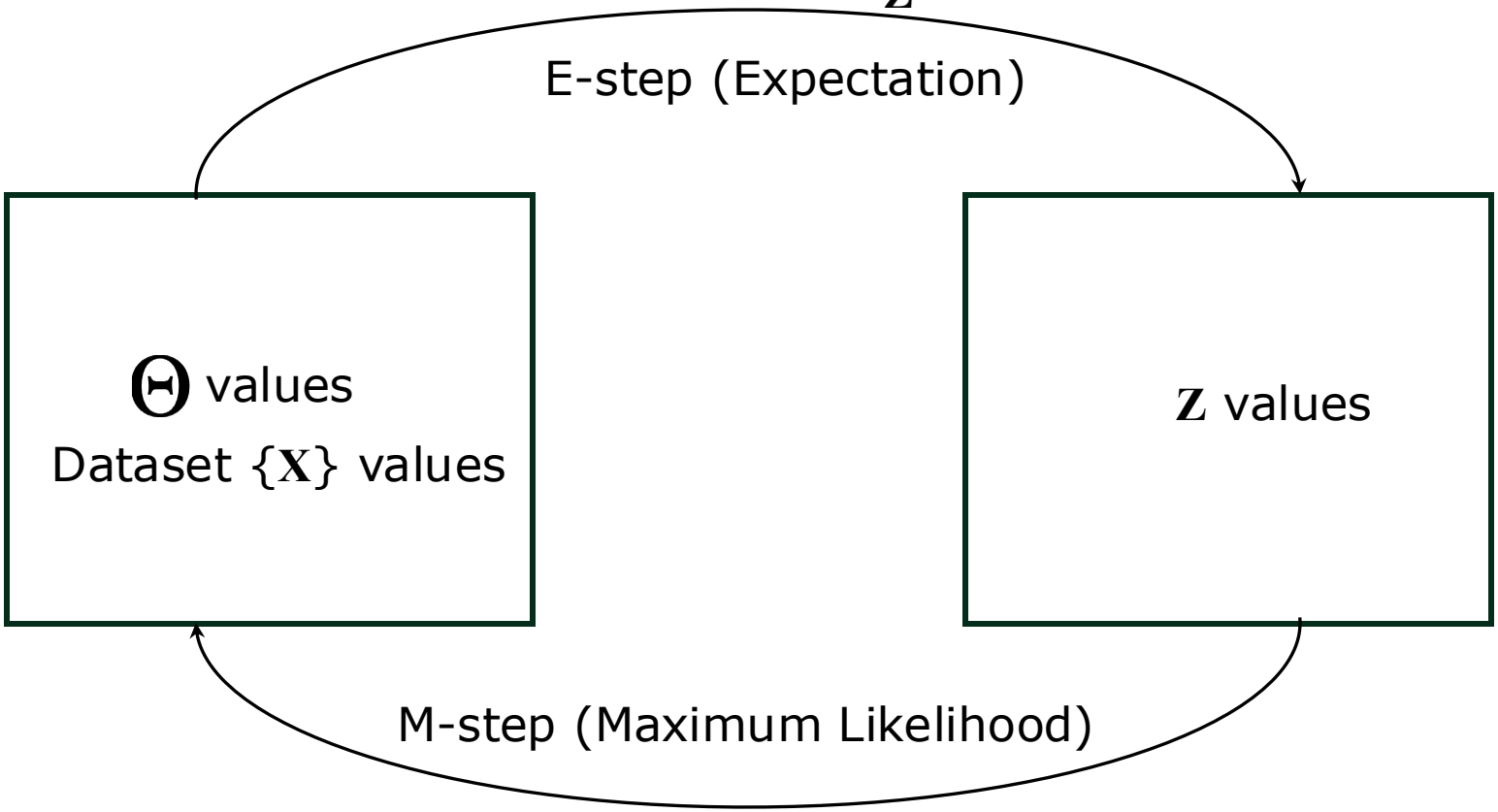
- Given the value of Θ, from the training data, we can infer the expected value for each latent variable in $\mathbf{Z}$ (the E-step))
- Given the values of the latent variables in $\mathbf{Z}$, we can estimate Θ with the maximum likelihood estimation (the M-step).

We initialize (7.35) with Θ^0 and iteratively apply the following two steps until convergence: :

- Infer the expectation of $\mathbf{Z}$ with Θ^t, denoted by $\mathbf{Z}^t$;
- Estimate Θ with the maximum likelihood estimation based on $\mathbf{Z}^t$ and the observed variables $\mathbf{X}$, denoted by Θ^{t+1} ;
- The above iterative process is the prototype of the EM algorithm.

EM Algorithm Diagram

$$LL(\Theta \mid \mathbf{X}) = \ln P(\mathbf{X} \mid \Theta) = \ln \sum_{\mathbf{Z}} P(\mathbf{X}, \mathbf{Z} \mid \Theta) \quad (7.35)$$



$$\underset{\Theta}{argmax}\{LL(\Theta \mid X, Z)\}$$

EM Algorithm

Instead of using the expectation of $\mathbf{Z}$, we can compute the probability distribution $P(\mathbf{Z} \mid \mathbf{X}, \Theta^t)$ based on Θ^t, and the above two steps become:

- E-step (Expectation): infer the latent variable distribution $P(\mathbf{Z} \mid \mathbf{X}, \Theta^t)$ based on Θ^t, and then compute the expectation of the log-likelihood $LL(\Theta \mid \mathbf{X}, \mathbf{Z})$ with respect to $\mathbf{Z}$ as:

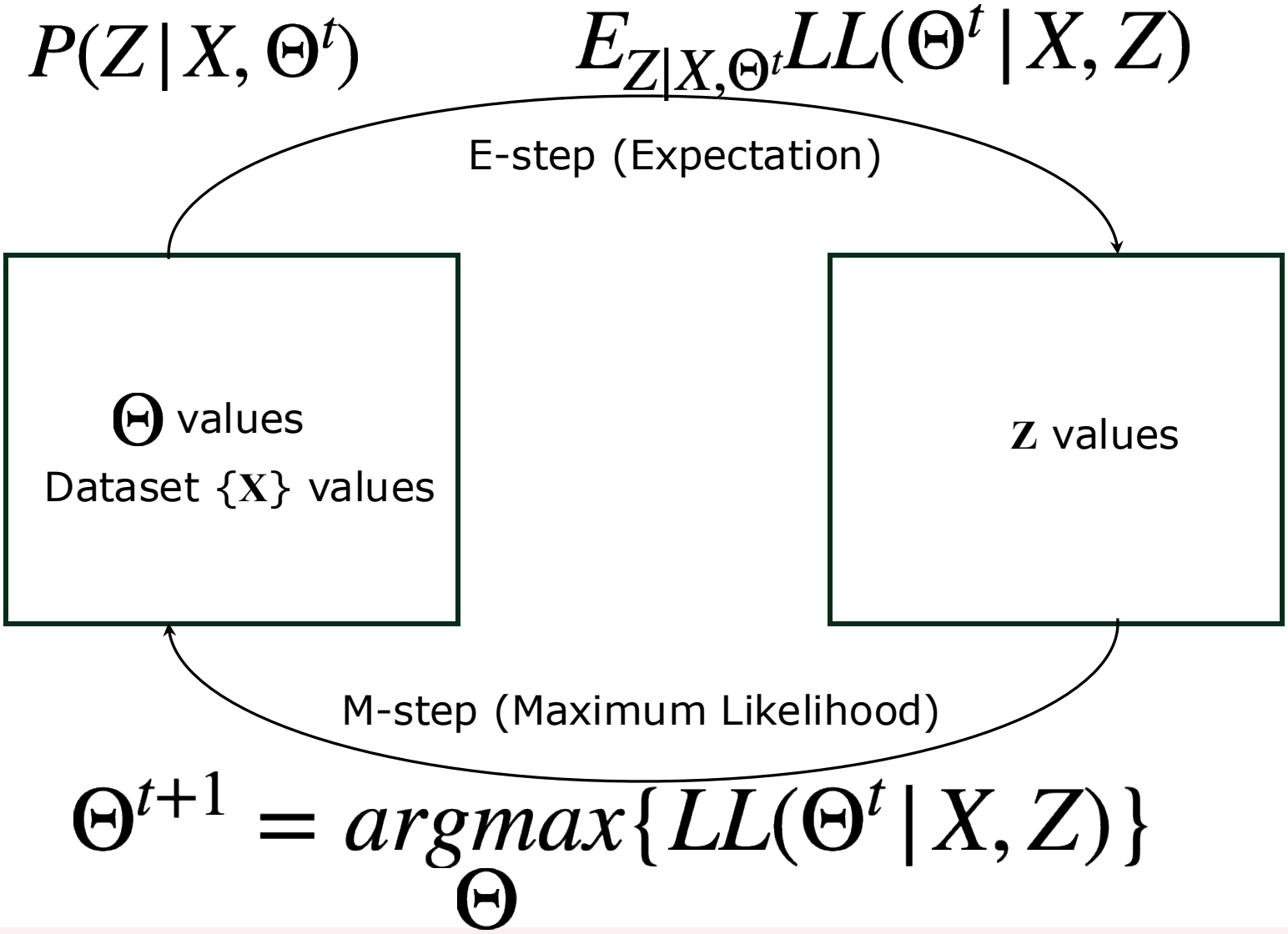
$$Q(\boldsymbol{\Theta} \mid \boldsymbol{\Theta}^t) = \mathbb{E}_{\mathbf{Z}|\mathbf{X},\boldsymbol{\Theta}^t} LL(\boldsymbol{\Theta} \mid \mathbf{X}, \mathbf{Z}) \qquad (7.36)$$

- M-step (Maximization): find the parameters that maximize the expected log-likelihood, that is,

$$\boldsymbol{\Theta}^{t+1} = \underset{\boldsymbol{\Theta}}{\operatorname{argmax}}\, Q(\boldsymbol{\Theta} \mid \boldsymbol{\Theta}^t) \qquad (7.37)$$

- EM algorithm finds the parameters that maximize the expected log-likelihood in the E-step. Iterating the above two steps until converging to a local optimum.

EM Algorithm Diagram (version 2)



Summary

- Bayes Decision Theory
- Maximum Likelihood Estimation
- Naïve Bayes Classifier
- Semi-Naïve Bayes Classifier
- Bayesian Network
- EM Algorithm

Thanks!